Lecture Notes:
Time & Space Complexity

Source: Code & Debug (DSA Python Course 2025)

1. Introduction

Understanding Time and Space complexity is **critical for software engineering interviews**. Regardless of the code written, an interviewer will almost always ask for the '**Approach**' followed by the '**Time and Space Complexity**'. It is the standard metric for measuring the efficiency of an algorithm.

2. What is Time Complexity?

Common Misconception:

Many beginners falsely define Time Complexity as *'The time taken (in seconds) for the code to run'*. This is **incorrect** because execution time depends on hardware (e.g., a Mac vs. an old Pentium machine) and background processes.

Correct Definition:

Time Complexity is the Rate of Increase in time with respect to the Input Size.
It measures how the number of operations grows as the input data (N) grows.

3. Big O Notation (The 3 Golden Rules)

The industry standard for expressing Time Complexity is **Big O Notation**.

Rule 1: Always Calculate for the Worst Case

- **Best Case (Omega Ω):** Minimum operations.
- **Average Case (Theta Θ):** Expected operations.
- **Worst Case (Big O):** Maximum operations. (We care about this to prevent server crashes under heavy load).

Rule 2: Avoid/Ignore Constant Values

If complexity is $O(3N^2 + 15)$, we drop the constants. For very large inputs, adding 15 or multiplying by 3 is negligible.

Result: $O(N^2)$

Rule 3: Avoid/Ignore Lower Bounds

If an equation is $O(N^2 + N)$, we keep only the term with the highest power.
Result: $O(N^2)$

4. Calculation Examples

Example 1: Simple Linear Loop

```
for i in range(1, N+1):
    print('Aniruddha')
```

Analysis:

- Operations per iteration: Check condition, Print, Increment
- Total: 3 operations × N iterations = 3N
- **Complexity: O(N)** (Linear Time)

Example 2: Independent Nested Loops

```
for i in range(1, N+1):      # Runs N times
    for j in range(1, N+1):  # Runs N times
        # Some operation
```

Analysis:

- Total Operations: $N \times N = N^2$
- **Complexity: $O(N^2)$** (Quadratic Time)

Example 3: Dependent Nested Loops

```
for i in range(1, N+1):
    for j in range(1, i):    # Depends on 'i'
```

Analysis:

- Inner loop runs: 1 + 2 + 3 + ... + N times
- Sum Formula: $N(N+1)/2 = (N^2 + N)/2$
- Dropping constants and lower bounds $\rightarrow$ **$O(N^2)$**

5. Space Complexity

Definition: The total memory space required by the code to execute.

Formula: Total Space = Auxiliary Space + Input Space

- **Input Space:** Memory to store the input data.
- **Auxiliary Space:** Extra temporary memory used by the algorithm.

Important Rule: Do not modify the input data to save space unless explicitly permitted. Always use auxiliary variables.

Example:

```
# Creating a copy of a list
new_list = []
for item in input_list:
    new_list.append(item)
```

Since 'new_list' grows as N grows, the **Space Complexity is O(N)**.

6. Quick Reference Cheat Sheet

Notation	Name	Description
Best Case (Ω)	Omega	Lower Bound (Minimum time)
Average Case (Θ)	Theta	Expected time
Worst Case (O)	Big O	Upper Bound (Maximum time) Used in Interviews

Complexity Trend (Fast to Slow):

$$O(1) \rightarrow O(\log N) \rightarrow O(N) \rightarrow O(N^2) \rightarrow O(2^N)$$